🤖 NDT-Based Robot Localization from LiDAR Scans

This project implements a **Normal Distributions Transform (NDT)** localization system in Python. It allows a robot to determine its precise position on a 2D map by matching real-time LiDAR sensor data against a statistically modeled environment.

🚀 Project Overview

Localization is a fundamental challenge in robotics. While robots use wheel encoders (odometry) to estimate movement, these sensors accumulate error over time due to slippage. This project solves that by "snapping" LiDAR scans onto a pre-existing map using NDT.

Instead of matching individual pixels, NDT divides the map into a grid and calculates the **Mean** and **Covariance** of obstacles in each cell. This creates a smooth "probability surface" that a mathematical optimizer can use to find the best fit for the robot's sensor data.

🛠️ How It Works

1. Map Pre-processing

The system loads a raw image map and uses **HSV Color Filtering** to remove non-obstacle data (like green grid lines). It then converts the result into a binary obstacle map where white pixels represent walls.

2. NDT Grid Generation

The map is partitioned into cells (e.g., 20×20 pixels). For each cell, we calculate:

- **Mean (μ):** The center of gravity of the wall pixels.
- **Covariance (Σ):** The orientation and "spread" of the wall.

This turns a jagged pixel map into a collection of Gaussian distributions.

3. LiDAR Scan Matching

As the robot moves, it collects distance measurements. The algorithm projects these points onto the map and calculates a **Match Score** using the multivariate normal distribution formula:

$$\text{Score} = \sum \exp \left(-\frac{1}{2} (\mathbf{x}_i - \mu)^T \Sigma^{-1} (\mathbf{x}_i - \mu) \right)$$

4. Pose Optimization

The system uses a **Hill-Climbing Optimizer**. It slightly nudges the robot's estimated position (x, y, θ) in multiple directions and keeps the pose that results in the highest NDT score.

📂 **Repository Structure**

- hello-3.c: (Optional) Example of kernel-level memory management using __init macros.
- main.py: The core Python script containing the NDT pipeline.
- aces_relations.png: The input map image.
- aces.clf: The CARMEN Log Format dataset containing LiDAR scans.
- cleaned_map.png: The output of the image processing phase.
- final_trajectory.png: The final visualization of the tracked path.

---

efore the robot can navigate, it needs to know what is a "wall" and what is just "noise."

- **The Problem:** Many digital maps have grid lines, labels, or colored zones (like green areas) that aren't physical obstacles.
- **The Solution:** The code converts the image to **HSV (Hue, Saturation, Value)**. It identifies the specific "Green" spectrum and replaces those pixels with white. This ensures the robot doesn't try to "detect" a green floor as a solid wall.

---

2. The Math: Normal Distributions Transform (NDT)

Instead of comparing the laser points to millions of tiny pixels (which is slow), the code uses **NDT**.

- **The Grid:** It divides the map into 20×20 pixel cells.
- **Statistical Modeling:** For each cell, it calculates:
  - **Mean (μ):** The "average" center point of the wall in that cell.
  - **Covariance (Σ):** The "shape" of the wall (Is it a vertical line? A horizontal one? A corner?).
- **Why?** This turns a "jagged" pixel map into a "smooth" mathematical surface. It's much easier for a robot to calculate its position against a smooth curve than a pixelated edge.

---

3. Data Parsing: The LiDAR "Brain"

The code reads a .clf file, which is a log of a real robot's journey.

- **Laser Ranges:** The distances to the nearest objects in a 180° arc.
- **Odometry Pose:** The robot's "guess" of where it is, based only on its wheel rotations.
- **The Catch:** Wheels slip and skid. After 10 minutes of driving, the "guess" is usually off by meters. This code exists to fix that error.

---

4. The Optimizer: "Hill Climbing"

This is the core "Correction" loop. For every new laser scan, the robot performs these steps:

1. **Project:** It takes the laser distances and draws them on the map where it *thinks* it is.
2. **Score:** It checks the NDT grid. If the laser points land right on the "Mean" of the wall cells, it gets a high score.
3. **Nudge:** It tries moving its guess slightly (5cm left, 5cm right, 2° rotation).
4. **Update:** If a nudge results in a higher score (better alignment), it updates its position to that nudge.

---

5. Fast-Forward Simulation

To make the code run quickly, it doesn't process every single millisecond of data.

- It **skips 10 steps** at a time.
- It uses the wheel data (Odometry) to move between skips.
- It uses the **NDT Optimization** to "snap" the robot back to the correct position every time it drifts.